Table of content

Preamble

The present document provides some information about the HDL code available in the 02 Bitbucket directory. The code may not be optimal or elegant, but it passes all the tests supplied in the Nand2Tetris project files.

`HalfAdder` chip

As indicated in the project documentation, this chip can be built using `Xor` and `And` chips. And so, the resulting HDL code is simple as :

```
Xor(a=a, b=b, out=sum);
And(a=a, b=b, out=carry);
```

`FullAdder` chip

The project documentation states that the chip can be implemented using 2 Half-Adders plus 1 simple gate. Indeed, one can check that the following code passes the supplied test :

```
HalfAdder(a=a, b=b, sum=sab, carry=cab);
HalfAdder(a=sab, b=c, sum=sum, carry=c1);
Xor(a=cab, b=c1, out=carry);
```

`Add16` chip

The code that can be found into `Add16.hdl` is a direct application of the project documentation : we first compute the sum of the least significant pair of bits using a Half-Adder and then we propagate the carry bit into the addition of the next pair of bits. And we repeat this process until the 15th pair of bits :

```
HalfAdder(a=a[0],  b=b[0],          sum=out[0],  carry=c0);
FullAdder(a=a[1],  b=b[1],  c=c0,   sum=out[1],  carry=c1);
FullAdder(a=a[2],  b=b[2],  c=c1,   sum=out[2],  carry=c2);
(...)
FullAdder(a=a[15], b=b[15], c=c14, sum=out[15], carry=c15);
```

`Inc16` chip

This chip adds `1` to a 16-bits number and it does not detect a possible overflow. The addition itself is simply done by using the previous chip but how can we define the integer constant `1` ? Looking back into the HDL code (written more than 3 years ago), I've seen that a specific pin (`one`) has been created :

```
Or16(a=false, b[0]=true, b[1..15]=false, out=one);
Add16(a=in, b=one, out=out);
```

This probably makes the code more readable but there's a faster (and hence better !) way to achieve the same computation :

```
Add16(a=in, b[0]=true, b[1..15]=false, out=out);
```

`ALU` chip

This chip computes the 18 functions described in the project documentation. Some comments can be found in the HDL file. Again, the proposed solution may not be optimal but it passes the 2 supplied tests (`ALU-nostat.tst` and `ALU.tst`).

Helper scripts

The boring task of writing this kind of repetitive lines of code can be avoided by using helper scripts :

```
FullAdder(a=a[3], b=b[3], c=c2, sum=out[3], carry=c3);
FullAdder(a=a[4], b=b[4], c=c3, sum=out[4], carry=c4);
(...)
FullAdder(a=a[15], b=b[15], c=c14, sum=out[15], carry=c15);
```

3 examples of Windows batch scripts (`genCode1[2,3].bat`) are available in the Bitbucket repository.